

Numpy

安装

```
1 pip install numpy
```

导入

```
1 import numpy as np
```

构建numpy对象

从数组构建

```
1 array=np.array([1,2,3,4,5])
```

构造多维数组

```
1 array=np.array([[1,2,3],[4,5,6]])
```

构建等差数列

```
1 array=np.arange(0,100,10) #0到100,step=10
```

构建时指定类型

```
1 mask=np.array([0,0,0,1,1,1,0,0,1,1],dtype=bool)
```

构造随机数数组

```
1 random_array=np.random.rand(10)
```

任何元素数据类型

```
1 array_any=np.array([1,10,3.5,'str'],dtype=np.object)
```

根据源对象构建新类型对象

```
1 #以下2种方法均不改变原来对象
2 ori_array=np.array([1,2,3])
3 new_array=np.asarray(ori_array,dtype=np.float32)
4 #或者
5 new_array=ori_array.astype(np.float32)
6
```

生成构建

arange

```
1 #含头不含尾
2 np.arange(10)
3 np.arange(1,10)
4 np.arange(1,10,2)
5 np.arange(1,10,2,dtype=np.int32)
```

linespace等间距构建

```
1 #指定区间构造n个数
2 np.linspace(min,max,n)
3
4 #0-10之间构造50个数
5 #包含头尾
6 np.linspace(0,10,50)
```

logspace等比构建

```
1 #0-10之间构造5个数
2 #包含头尾
3 np.logspace(0,10,5)
```

zero构建

```
1 #构建后以0填充
2 np.zeros(3)
3 np.zeros(3,3)
4 #从模板构建
5 np.zeros_like(arr)
```

ones构建

```
1 #构建后以1填充
2 np.ones(3)
3 np.ones((3,3))
4 np.ones((3,3))*2
5 np.ones((3,3),dtype=np.float32)
6 #从模板构建
7 np.ones_like(arr)
```

empty构建

```
1 #非初始化值
2 arr=np.empty(10)
3 arr.fill(1)
```

构建单位矩阵

```
1 np.identity(5)
```

构建网格

```
1 x=np.array([1,2,3,4,5])
2 y=x-5
3 x,y=np.meshgrid(x,y)
```

行列向量构建

```
1 row=np.r_[0:10:1] #行向量 [0 1 2 3 4 5 6 7 8 9]
2 col=np.c_[0:10:1] #列向量  [[0]
3                               #   [1]
4                               #   [2]
5                               #   [3]
6                               #   [4]
7                               #   [5]
8                               #   [6]
9                               #   [7]
10                              #   [8]
11                              #   [9]]
```

修改操作

填充

```
1 array.fill(0)
```

浅拷贝

```
1 arr2=arr1
```

深拷贝

```
1 arr2=arr1.copy()
```

值限制

```
1 arr=np.array([1,2,3,4,5])
2 arr.clip(2,4) #结果[2,2,3,4,4]
```

近似值四舍五入

```
1 arr=np.array([1.2,3.56,6.41])
2 arr.round() #结果[1,4,6] , 0位小数
3 arr.round(decimals=1) #结果[1,4,6] , 1位小数
```

数值形状操作

```
1 array=np.arange(10)
2 array.shape=2,5 #修改为2行5列,直接修改源对象
3 array.reshape(2,5) #产生新对象,源对象不变
```

增加维度

```
1 array=np.arange(10)
2 array.shape #(10,)
3 array=array[np.newaxis,: ]
4 array.shape #(1,10)
```

减少维度

```
1 array.squeeze() #生成新对象
2 a.flatten() #转换为一维数值
3 a.ravel() #转换为一维数值
```

转置

```
1 array.transpose() #生成新对象
2 array.T
```

数组连接

```
1 a=np.array([[1,1],[2,2]])
2 b=np.array([[3,3],[4,4]])
3 c=np.concatenate((a,b))
4 print(c) # [1,1]
5           # [2,2]
6           # [3,3]
7           # [4,4]
```

等价于

```
1 c=np.vstack((a,b))
```

```
1 c=np.concatenate((a,b),axis=1)
2 print(c) # [1,1,3,3]
3           # [2,2,4,4]
```

等价于

```
1 c=np.hstack((a,b))
```

运算

与常量运算

```
1 array+=1
```

与np对象运算

```
1 arr1+arr2
```

返回数据位置

```
1 array=np.linspace(0,10,10)
2 values=np.array([2.5,6.5,9.5])
3 index=np.searchsorted(array,values) #[3,6,9]
```

集合运算

```
1 arr1=np.array([1,2,3])
2 arr2=np.array([1,2,4])
```

交集

```
1 print(np.intersect1d(arr1,arr2))
```

并集

```
1 print(np.union1d(arr1,arr2))
```

差集

```
1 print(np.setdiff1d(arr1,arr2))
```

all&any

```
1 a=np.array([1,2,3,4])
2 b=np.array([1,2,3,5])
3 np.all(a==b) #完全相等返回True否则返回False
4 np.any(a==b) #任一相等返回True否则返回False
```

统计运算

求和

```
1 array=np.array([[1,2,3],[4,5,6]])
2 np.sum(array)
```

根据维度求和

```
1 np.sum(array,axis=0) #[5,7,9] 按照列求和
2 array.sum(axis=0)    #[5,7,9] 按照列求和
3 np.sum(array,axis=1) #[6,15] 按照行求和
4 array.sum(axis=1)    #[6,15] 按照行求和
5 np.sum(array,axis=-1)#[6,15] 按照最后一个轴求和
```

求积

```
1 array.prod()
2 array.prod(axis=0)
3 array.prod(axis=1)
```

最小值

```
1 array.min()  
2 array.min(axis=0)  
3 array.min(axis=1)
```

最大值

```
1 array.max()  
2 array.max(axis=0)  
3 array.max(axis=1)
```

最小值索引

```
1 array.argmin()  
2 array.argmin(axis=0)  
3 array.argmin(axis=1)
```

最大值索引

```
1 array.argmax()  
2 array.argmax(axis=0)  
3 array.argmax(axis=1)
```

均值

```
1 array.mean()  
2 array.mean(axis=0)  
3 array.mean(axis=1)
```

标准差

```
1 array.std()  
2 array.std(axis=0)  
3 array.std(axis=1)
```

方差

```
1 array.var()  
2 array.var(axis=0)  
3 array.var(axis=1)
```

四则运算

```
1 x=np.array([5,5])  
2 y=np.array([2,2])
```

乘法

```
1 np.multiply(x,y) #[10,10] 对应位置相乘  
2 np.dot(x,y)      #矩阵乘法  
3 x=np.array([1,1,1])  
4 y=np.array([[1,2,3],[4,5,6]])  
5 x*y              #[[1,2,3],[4,5,6]],x扩充维度然后对应位作乘法
```

比较运算

```
1 x==y    #[False,False],对应位比较，维度需要一样。
```

逻辑运算

and运算

```
1 np.logical_and(x,y)
```

or运算

```
1 np.logical_or(x,y)
```

not运算

```
1 np.logical_not(x)
```

索引与切片

```
1 arr[3]    #取第4个数据
2 arr[m:n]  #切片
3 arr[-2]   #取倒数第二个数据
4 arr[-2:]  #取最后2个数据
```

多维数组索引

```
1 array=np.array([[1,2,3],[4,5,6],[7,8,9]])
2 array[1,1]    #获取元素5
3 array[1,1]=10 #修改元素5为10
4 array[1]      #获取第二行[4,5,6]
5 array[:,1]    #获取第列[2,5,8]
6 array[0,0:2]  #获取第一行元素[1,2]
```

where检索数据

```
1 array=np.array([10,20,30,40,50])
2 np.where(array>30)          #获取大于30的元素的下标 [3,4]
3 array[np.where(array>30)]   #获取大于30的元素 [40,50]
4 array[array>30]
```

通过布尔对象索引数据

```
1 array=np.arange(0,100,10) #0到100,step=10
2 mask=np.array([0,0,0,1,1,1,0,0,1,1],dtype=bool)
3 array[mask] #结果[30,40,50,80,90]
```

属性获取

```
1 #获取数据结构
2 array.shape=>(x,y)
3 np.shape(array)
4 #获取数据类型
5 array.dtype
6 #查看每个原始占用字节数
7 print(array.itemsize)
8 #查看元素个数
9 print(array.size)
10 np.size(array)
11 #查看维度
12 print(array.ndim)
13 #查看数据占用字节数
14 print(array.nbytes)
```

排序操作

```
1 arr=np.array([1.5,1.3,7.5],
2              [5.6,7.8,1.2])
3
4 np.sort(arr)          #按行排序
5 np.sort(arr,axis=0)   #按列排序
6 np.sort(arr,axis=1)   #按行排序
```

返回原来的索引值

```
1 np.argsort(arr)
```

多个列联合排序

```
1 #第三列升序，第一列降序
2 array=np.array([[1,0,6],
3                 [1,7,0],
4                 [2,3,1],
5                 [2,4,0]])
6 index=np.lexsort([-1*array[:,0],array[:,2]])
7 print(array[index])
```

随机数

普通数据生成

```
1 np.random.rand(3,2)          #3X2的随机数，值从0到1
2 np.random.rand()             #返回一个随机数，值从0到1
3 np.random.random_sample()     #返回一个随机数，值从0到1
4 np.random.randint(10,size=(3,2)) #3X2的随机整数，值从0到10，左闭右开
5 np.random.randint(0,10,3)     #0-10中返回3个数
```

高斯分布数据

```
1 mu,sigma=0,0.1
2 n=np.random.normal(mu,sigma,10) #正态分布构建,10个值
```

洗牌操作

```
1 arr=np.arange(10)
2 np.random.shuffle(arr) #修改源对象
```

随机种子指定

```
1 #指定种子后,可保证随机的模式是一样的
2 np.random.seed(10)
```

文件操作

从文件加载

```
1 -----data.txt-----
2 1 2 3 3 4
3 5 5 6 7 8
4 -----
5 arr=np.loadtxt('data.txt') #以空格为分隔符
6 arr=np.loadtxt('data.txt',delimiter=',') #以逗号为分隔符
7 arr=np.loadtxt('data.txt',delimiter=',',skiprows=1) #以逗号为分隔符,跳过第一行
8 arr=np.loadtxt('data.txt',delimiter=',',usecols=(0,1,4)) #指定使用列
```

保存数据到文件

```
1 arr=np.random.randint(-10,10,size=(10,10))
```

读取与保存数据

```
1 #逗号分隔，保存为整数
2 np.savetxt('data.txt',arr,fmt="%d",delimiter=",")
3 #逗号分隔，保存为浮点数，2位小数
4 np.savetxt('data.txt', arr, fmt="%.2f", delimiter=",")
```

读取与保存ndarray结构

```
1 #保存ndarray结构
2 np.save('data.npy',arr)
3 #读取ndarray结构
4 arr=np.load('data.npy')
```

批量保存与读取ndarray数据

```
1 arr1 = np.random.randint(-10, 10, size=(10, 10))
2 arr2 = np.random.randint(-10, 10, size=(10, 10))
3 #批量保存
4 np.savez('data.npz',a=arr1,b=arr2)
5 #批量读取
6 data=np.load('data.npz')
7 #keys查看
8 data.keys()
9 arr1=data['a']
10 arr2=data['b']
11 print(arr1)
12 print(arr2)
```

其他

设置小数精度

```
1 np.set_printoptions(precision=3)
```

打印帮助文档

```
1 print(help(np.pad))
```

设置打印限制

```
1 np.set_printoptions(threshold=5)
2 np.set_printoptions(threshold=sys.maxsize) #打印全部
```

练习题

1. 打印版本号

```
1 print(np.version)
```

2. 构建一个全0的矩阵，并打印其占用内存大小

```
1 arr=np.zeros((3,3))
2 print(arr.nbytes)
```

3. 创建一个10-49的数值,并倒序排列

```
1 arr=np.arange(10,50,1)
2 arr=arr[::-1]
3 print(arr)
```

4. 找一个数值中不为0的索引

```
1 np.nonzero([1,2,3,4,0,0,1])
```

5. 随机构造一个33的矩阵，并打印其最大值与最小值

```
1 arr=np.random.randint(-10,10,size=(3,3))
2 print(f'max={np.max(arr)} min={np.min(arr)}')
```

6. 构造一个55的矩阵，让其值都是1，并在最外层加上一圈0

```
1 arr=np.ones((5,5))
2 arr=np.pad(arr,pad_width=1,constant_values=0)
3 print(arr)
```

7. 构建一个shape为(6,7,8)的矩阵，并找到第100个元素的索引和值

```
1 arr=np.random.randint(0,1000,size=(6,7,8))
2 index=np.unravel_index(100,arr.shape)
3 print(f'index is {index} value is {arr[index]}')
```

8. 对一个5*5的矩阵做归一化操作

```
1 arr=np.random.randint(0,1000,size=(5,5))
2 max=arr.max()
3 min=arr.min()
4 arr=(arr-min)/(max-min)
5 print(arr)
```

9. 找到两个数值中相同的值

```
1 arr1=np.random.randint(0,120,size=(100))
2 arr2=np.random.randint(0,120,size=(100))
3 print(np.intersect1d(arr1,arr2))
4 print(arr1[arr2==arr1])
```

10. 得到今天,明天,昨天的日期

```
1 today = np.datetime64('today', 'D')
2 yesterday = today - np.timedelta64(1, 'D')
3 tomorrow = today + np.timedelta64(1, 'D')
4 print(today)
5 print(yesterday)
6 print(tomorrow)
```

11. 得到一个月的所有天

```
1 days=np.arange('2017-10','2017-11',dtype='datetime64[D]')
2 print(days)
```

12. 得到一个数的整数

```
1 z=np.random.uniform(0,10,10)
2 print(np.floor(z))
```

13. 构造一个数值, 让他不能改变

```
1 z=np.zeros(5)
2 z.flags.writeable=False
```

14. 找到在一个数值中，最接近一个数的索引

```
1 z=np.arange(100)
2 v=np.random.uniform(0,100)
3 print(v)
4 index=(np.abs(z-v)).argmin()
5 print(z[index])
```

15. 32位float类型和32位int类型转换

```
1 z=np.arange(10,dtype=np.int32)
2 z=z.astype(np.float32)
```

16. 打印数组元素位置坐标与数值

```
1 z = np.arange(9).reshape(3, 3)
2 for index, value in np.ndenumerate(z):
3     print(f'value:{index}---{value}')
```

17. 按照数组的某一列进行排序

```
1 a=np.random.randint(0,100,(3,3))
2 print(z[z[:,1].argsort()])
```

18. 统计数值中每一个值出现的次数

```
1 z = np.array([1, 1, 1, 2, 2, 3, 3, 3, 3, 8])
2 print(np.bincount(z))
```

19. 对一个四维数值的最后两维求和

```
1 z=np.random.randint(0,10,(4,4,4,4))
2 res=z.sum(axis=(-2,-1))
3 print(res)
```

20. 交换矩阵中的两行

```
1 z=np.arange(25).reshape(5,5)
2 z[[0,1]]=z[[1,0]]
```

21. 找到数值中最常出现的一个数

```
1 z=np.random.randint(0,10,50)
2 print(np.bincount(z).argmax())
```

22. 快速查找topk

```
1 z=np.arange(10000)
2 np.random.shuffle(z)
3 n=5
4 print(z[np.argpartition(-z,n)[:n]])
```

Pandas

数据分析处理

<https://www.py pandas.cn/>

<https://pandas.pydata.org/>

导入

```
1 import pandas as pd
```

构建DataFrame

从CSV文件读取

```
1 #直接读取
2 df=pd.read_csv('file.csv')
3 #指定编码
4 df=pd.read_csv("d:/tmp/pandas/file.csv",encoding="gbk")
```

手工构造

```
1 #方法1 数据对象构造
2 data={
3     '国家':['GJA','GJB','GJC'],
4     '人口':[100,200,300]
5 }
6 df=pd.DataFrame(data)
7
8 #方法2 参数构造
9 df = pd.DataFrame([[1, 2, 3], [4, 5, 6]], index=['a', 'b'], columns=['A', 'B', 'C'])
```

信息展示

```
1 #显示前几行
2 df.head()
3 df.head(10)
4 #返回当前对象信息
5 df.info()
6 #显示索引值
7 df.index
8 #显示列
9 df.columns
10 #显示列数据类型
11 df.dtypes
12 #显示数据
13 df.values
14 #显示统计指标
15 print(df.describe())
```

数据查询与检索

查询指定列

```
1 df['Age']
```

查询指定列前n行

```
1 age=df['Age']
2 age[:5]
```

根据指定列Series检索

```
1 df=df.set_index("name")
2 age=df['age']
3 print(age['name01'])
4
5 #可以指定多列数据
6 rowdata=df[['age','sex']]
7 print(rowdata['name01'])
```

数据位置检索

```
1 print(data.iloc[0:5])      #返回所有列
2 print(data.iloc[0:5,1:2]) #返回指定列
3 print(data.iloc[0:5,0])   #返回指定列
```

标签检索

```
1 print(data.loc['name01'])      #返回所有列
2 print(data.loc['name01','age']) #返回指定列
3 print(data.loc['name01':'name08',:]) #返回范围
```

bool检索

```
1 #查看年龄大于50的行
2 print(df[df['age']>50])
3 #性别为male的人的平均年龄
4 df.loc[df['Sex']=='male','Age'].mean()
5 #年龄大于70的人的数量
6 (df['Age']>70).sum()
```

修改操作

设置索引列

```
1 df=df.set_index('Name')
```

修改列值

```
1 ;#erias修改
2 age=df['age']
3 age=age+10
4 #loc定位修改
5 data.loc['name01','age']=80
```

对象操作

Series对象操作

```
1 data=[10,11,12]
2 index=['a','b','c']
3 s=pd.Series(data=data,index=index)
4
5 #索引与切片
6 s[0]    #索引
7 s[1:2]  #切片
8
9 #bool索引
10 s[mask] #bool索引
11
12 #根据索引查询
13 s.loc['a']
14
15 #根据位置查询
16 s.iloc[1]
17
18 #深度拷贝
19 s1=s.copy()
20
21 #修改值
22 s1['a']=100
23
24 #100改为101,inplace=True在源对象上修改。
25 s1.replace(to_replace=100,value=101,inplace=True)
26
27 #修改索引
28 s1.index=['a','b','d']
29
30 #修改指定索引
31 s1.rename(index={'a':'A'},inplace=True)
32
33 #Series结构的删除
34 del s['A'] #删除单个数据
35 s.drop(['b','d'],inplace=True) #删除多个数据
```

DataFrame对象操作

检索数据

```
1  #根据索引查询
2  df['A']
3  df.loc['A']
4
5  #根据位置查询
6  df.iloc[0]
7
8  #修改操作
9  df.loc['a']['ColumnName']=value
10
11 #修改索引
12 df.index=['f','g']
13
14 #增加操作
15 df.loc['c']=[1,2,3]
16
17 #拼接操作
18 df3=pd.concat([df,df2],axis=0)
19 df3=pd.concat([df,df2],axis=1)
20
21 #增加列
22 df['NewColumnName']=[10,11]
23
24 #增加多列
25 dfadd=pd.DataFrame([[10,11],[12,13]],index=['j','k'],columns=['D','E'])
26 df5=pd.concat([df2,dfadd],axis=1)
27
28 #删除操作
29 df.drop(['j'],axis=0,inplace=True)      #删除行
30 del df['Tang']                          #删除列
31 df.drop(['A','B','C'],axis=1,inplace=True) #删多列
```

merge操作

单keymerge

```
1 left=pd.DataFrame({
2     'key':['K0','K1','K2','K3'],
3     'A':['A0','A1','A2','A3'],
4     'B':['B0','B1','B2','B3']
5 })
6
7 right=pd.DataFrame({
8     'key':['K0','K1','K2','K3'],
9     'C':['C0','C1','C2','C3'],
10    'D':['D0','D1','D2','D3']
11 })
12
13 pd.merge(left=left,right=right)
14 pd.merge(left=left,right=right,on='key')
```

多Key Merge

```
1 left=pd.DataFrame({
2     'key1':['K0','K1','K2','K3'],
3     'key2':['K0','K1','K2','K3'],
4     'A':['A0','A1','A2','A3'],
5     'B':['B0','B1','B2','B3']
6 })
7
8 right=pd.DataFrame({
9     'key1':['K0','K1','K2','K3'],
10    'key2':['K0','K1','K2','K4'],
11    'C':['C0','C1','C2','C3'],
12    'D':['D0','D1','D2','D3']
13 })
14
15 pd.merge(left=left,right=right)
16 pd.merge(left=left,right=right,on=['key1','key2']) #交集合并
17 pd.merge(left=left,right=right,on=['key1','key2'],how='outer') #并集合并
18 pd.merge(left=left,right=right,on=['key1','key2'],how='outer',indicator=True) #添加指示
19 pd.merge(left=left,right=right,on=['key1','key2'],how='left') #指定基准表
20 pd.merge(left=left,right=right,on=['key1','key2'],how='right') #指定基准表
```

其他操作

字符串操作

```
1 s=pd.Series(['A','B','b','gaer','AGER',np.nan])
2 s.str.lower()
3 s.str.upper()
4 s.str.len()
5
6 index=pd.Index([' tang',' yu ','di'])
7 index.str.strip()
8 index.str.lstrip()
9 index.str.rstrip()
10
11 s=pd.Series(['a_b_c','c_d_e','f_g_h'])
12 s.str.split('_')
13 s.str.split('_',expand=True)
14 s.str.split('_',expand=True,n=1)
15 s.str.contains('Aa')
16
17 s=pd.Series(['a','a|b','a|c'])
18 s.str.get_dummies(sep='|')
```

修改列名

```
1 df.columns.str.replace(' ','_')
```

大数据处理

其他设置

显示设置

```
1 pd.get_option('display.max_rows')      #获取最大显示行数
2 pd.set_option('display.max_rows',100) #设置最大显示行数
3
4
5 pd.get_option('display.max_columns')    #获取最大显示列数
6 pd.set_option('display.max_columns',100) #设置最大显示列数
7
8
9 pd.get_option('display.max_colwidth')   #获取最大显示字符串宽度
10 pd.set_option('display.max_colwidth',100) #设置最大显示字符串宽度
11
12
13 pd.get_option('display.precision')     #获取小数精度
14 pd.set_option('display.precision',3)   #设置小数精度
```

打印数据占用的内存信息

```
1 df.info(memory_usage='deep')
```

时间

普通时间操作

```
1 import datetime
2 dt=datetime.datetime(year=2017,month=11,day=24,hour=10,minute=30)
3 print(dt)
```

构造时间

```
1 #单个时间构造
2 ts=pd.Timestamp('2017-11-24')
3 pd.to_datetime('2017-11-24')
4 pd.to_datetime('24/11/2017')
5 #时间序列构造
6 s=pd.Series(['2017-11-24','2017-11-25','2017-11-26'])
7 ts=pd.to_datetime(s)
```

获取时间属性

```
1 ts.month          #获取月份
2 ts.day            #获取天
3 ts+pd.Timedelta('5 dasy') #延后5天
```

序列操作

```
1 #字符串构建
2 s=pd.Series(['2017-11-24','2017-11-25','2017-11-26'])
3 ts=pd.to_datetime(s)
4
5 #直接构建
6 ts=pd.Series(pd.date_range(start='2017-11-24',periods=10,freq='12H'))
7
8 #读取CSV后转换构建
9 data['Time']=pd.to_datetime(data['Time'])
10
11 #读取CSV直接转换
12 data=pd.read_csv('file.csv',index_col=0,parse_dates=True)
13
14 ts.dt.hour
15 ts.dt.weekday
```

时间切片

```
1 data[pd.Timestamp('2012-01-01 09:00'):pd.Timestamp('2012-01-10 09:00')]
2 data[('2012-01-01 09:00'):( '2012-01-10 09:00')]
3 data['2013']
4 data['2012-01':'2012-03']
5 data[data.index.month==1]
6 data[(data.index.hour>8) & (data.index.hour<12)]
7 data.between_time('08:00','12:00')
```

时间统计

```
1 #根据天求均值
2 data.resample('D').mean()
3 data.resample('3D').mean()
4
5 #根据天求最大
6 data.resample('D').max()
7 data.resample('3D').max()
8
9 #根据天求最小值
10 data.resample('D').min()
11 data.resample('3D').min()
```

运算

统计运算

```
1 #平均值
2 df['age'].mean()
3
4 #最大值
5 df['age'].max()
```

数值运算

```
1 df = pd.DataFrame([[1, 2, 3], [4, 5, 6]], index=['a', 'b'], columns=['A', 'B', 'C'])
2
3 #求和
4 df.sum()          #按列求和
5 df.sum(axis=0)    #按列求和
6 df.sum(axis=1)    #按行求和
7 df.sum(axis='columns') #按行求和
8
9 #均值
10 df.mean()
11
12 #最小值
13 df.min()
14
15 #最大值
16 df.max()
17
18 #中位数
19 df.median()
```

分组统计

```
1 df.groupby('key').sum()
2 df.groupby('key').aggregate(np.sum)
3 df.groupby('key').aggregate(np.mean)
4 df.groupby('sex')['Age'].mean()
5
6 #高级分组统计
7
8 #普通统计
9 df=pd.DataFrame({
10     'A':['foo','bar','foo','bar','foo','bar','foo','foo'],
11     'B':['one','one','two','three','two','two','one','three'],
12     'C':np.random.randn(8)*100,
13     'D':np.random.randn(8)*10
14 })
15
16 df.groupby(['A']).count()
17 df.groupby(['A','B']).count()
18
19 #关注指定Group
20 df=pd.DataFrame(
21     {
22     'X':['A','B','A','B'],
23     'Y':[1,2,3,4]
24 })
25
26 df.groupby(['X']).get_group('A')
27
28
29 #统计特性
30 group=df.groupby(['X','Y'])
31 group.describe()
32 group['C'].agg([np.sum,np.mean,np.std])
33 group['C'].agg({'res_sum':np.sum,
34                 'res_mean':np.mean,
35                 'res_std':np.std})
```

二元统计

```
1 df.cov()                #协方差
2 df.corr()               #相关系数
3 df['Age'].value_counts() #值个数
4 df['Age'].value_counts(ascending=True) #排序统计值个数
5 df['Age'].value_counts(ascending=True,bins=5) #分区间统计值个数
6 df['Age'].count() #计数
```

常用操作

```
1 data=pd.DataFrame({
2   'group':['a','a','a','b','b','b','c','c','c'],
3   'data':[4,3,2,1,12,3,4,5,7]
4 })
```

排序操作

```
1 ;group 降序，data升序，源对象操作
2 data.sort_values(by=['group','data'],ascending=[False,True],inplace=True)
```

去重操作

```

1 data=pd.DataFrame({'K1':['one']*3+['two']*4 ,
2 'k2':[3,2,1,3,3,4,4]
3 })
4
5 drop.drop_duplicates()          #基于所有字段去重
6 drop.drop_duplicates(subset='k1') #基于K1去重

```

转换操作

apply转换操作

```

1 data=pd.DataFrame({'food':['A1','A2','B1','B2','B3','C1','C2'] ,
2 'data':[1,2,3,4,5,6,7]
3 })
4
5 def food_map(series):
6     if series['food']=='A1' :
7         return 'A'
8     elif series['food']=='A2' :
9         return 'A'
10    elif series['food']=='B1' :
11        return 'B'
12    elif series['food']=='B2' :
13        return 'B'
14    elif series['food']=='B3' :
15        return 'B'
16    elif series['food']=='C1' :
17        return 'C'
18    elif series['food']=='C2' :
19        return 'C'
20
21 data['food_map']=data.apply(food_map,axis='columns')

```

map转换操作

```
1 food2Upper={
2     "A1":"A",
3     "A2":"A",
4     "B1":"B",
5     "B2":"B",
6     "B3":"B",
7     "C1":"C",
8     "C2":"C"
9 }
10
11 data['upper']=data['food'].map(food2Upper)
```

根据其他列计算

```
1 df=pd.DataFrame({
2     'data1':np.random.randn(5),
3     'data2':np.random.randn(5)
4 })
5 df2=df.assign(ration=df['data1']/df['data2'])
```

Series替换值

```
1 data=pd.Series([1,2,3,4,5,6,7,8,9])
2 data.replace(9,np.nan,inplace=True)
```

分段统计

```
1 ages=[15,18,20,21,22,34,41,52,63,79]
2 bins=[10,40,80]
3 bins_res=pd.cut(ages,bins)
4 pd.value_counts(bins_res)
```

添加标签的分段统计

```
1 group_names=['Young','Mille','Old']
2 res=pd.cut(ages,[10,20,50,80],labels=group_names)
3 pd.value_counts(res)
```

缺失值判断

```
1 df.isnull()
2 df.isnull().any() #按列查看是否有缺失值
3 df.isnull().any(axis=1) #按行查看是否有缺失值
```

缺失值填充

```
1 df.fillna(5)
```

查找缺失值

```
1 df[df.isnull().any(axis=1)]
```

索引高级

```
1 s=pd.Series(np.arange(5),index=np.arange(5)[::-1],dtype='int64')
2 s.isin([1,3,4])
3 s[s.isin([1,3,4])]
4 s[s>2] #条件筛选
5
6 s2=pd.Series(np.arange(6),index=pd.MultiIndex.from_product([[0,1],['a','b','c']]))
7 s2.iloc[s2.index.isin([(1,'a'),(0,'b')])]
8
9 dates=pd.date_range('20171124',periods=8)
10 df=pd.DataFrame(np.random.randn(8,4),index=dates,columns=['A','B','C','D'])
11 df.select(lambda x:x=='A',axis='columns') #拿A列
12 df.where(df<0)
13 df.where(df<0,-df)
14
15 df=pd.DataFrame(np.random.rand(10,3),columns=list('abc'))
16 df.query('(a<b) & (b<c)')
```

Matplotlib

参考

<https://www.matplotlib.net/>

导入

```
1 import matplotlib.pyplot as plt
2 #Jupyter 中引入
3 %matplotlib inline
```

绘制

折线图

基本绘制

```
1 plt.plot([1,2,3,4,5],[1,4,9,16,25])
```

添加label

```
1 plt.xlabel('xlabel')
```

添加label同时设置字体大小

```
1 plt.xlabel('xlabel',fontsize=10)
2 plt.ylabel('ylabel')
3 plt.plot([1,2,3,4,5],[1,4,9,16,25])
```

添加title

```
1 plt.plot([1,2,3,4,5],[1,4,9,16,25])
2 plt.title("title")
```

设置文本

```
1 #0,0 是坐标
2 plt.text(0,0,"text")
```

设置栅格

```
1 plt.grid(True)
```

设置注释

```
1 plt.annotate('comment',xy=(-5,0),xytext=(2.0,5),arrowprops=dict(headwidth=20,facecolor='black',shrink=0.5,headlength=20))
```

设置线条样式

```
1 plt.plot([1,2,3,4,5],[1,4,9,16,25], '--')
2 plt.plot([1,2,3,4,5],[1,4,9,16,25], ':')
3 plt.plot([1,2,3,4,5],[1,4,9,16,25], '.')
```

```
4 plt.plot([1,2,3,4,5],[1,4,9,16,25], '*')
5 plt.plot([1,2,3,4,5],[1,4,9,16,25], 'x')
6 plt.plot([1,2,3,4,5],[1,4,9,16,25], linestyle='-')
```

设置线条颜色

```
1 #颜色参数: r|g|b|c|m|y|k|w
2 plt.plot([1,2,3,4,5],[1,4,9,16,25], '-', color='r')
3 plt.plot([1,2,3,4,5],[1,4,9,16,25], 'r-')
4 plt.plot([1,2,3,4,5],[1,4,9,16,25], 'ro')
```

设置线条宽度

```
1 x=np.linspace(-10,10)
2 y=np.sin(x)
3 plt.plot(x,y,linewidth=2.0)
```

一图多线

```
1 x=np.arange(0,10,0.5)
2 plt.plot(x,x,'r--')
3 plt.plot(x,x**2,'bs')
4 plt.plot(x,x**3,'go')
```

或者写在一起

```
1 plt.plot(x,x,'r--',x,x**2,'bs',x,x**3,'go')
```

标注

```
1 plt.plot([1,2,3,4,5],[1,4,9,16,25], '--', marker='o')
2 plt.plot([1,2,3,4,5],[1,4,9,16,25], '--', marker='o', markerfacecolor='r', markersize=10)
```

先画图然后设置参数

```
1 line=plt.plot(x,y)
2 plt.setp(line,color='r',linewidth=2.0,alpha=0.5)
```

子图

```
1 x=np.linspace(-10,10)
2 y=np.sin(x)
3
4 #2(2行)1(1列)1(第1个图)
5 plt.subplot(211)
6 plt.plot(x,y,color='r')
7
8 #2(2行)1(1列)1(第2个图)
9 plt.subplot(212)
10 plt.plot(x,y,color='b')
```

条形图

```
1 x=np.arange(5)
2 y=np.random.randint(-5,5,5)
```

```
1 #构建一行两列图
2 fig,axes=plt.subplots(ncols=2)
```

垂直条形图

```
1 vBars=axes[0].bar(x,y,color='red')
```

水平条形图

```
1 hBars=axes[1].barh(x,y,color='red')
```

添加横线

```
1 axes[0].axhline(0,color='gray',linewidth=2)
2 axes[1].axvline(0,color='gray',linewidth=2)
```

根据数值进行风格设置

```
1 fig,ax=plt.subplots()
2 vBars=ax.bar(x,y,color='lightblue')
3 for bar,height in zip(vBars,y):
4     if height < 0 :
5         bar.set(edgecolor='darkred',color='green',linewidth=3)
6 plt.show()
```

误差棒

```
1 mean_values=[1,2,3]
2 variance=[0.2,0.4,0.5]
3 bar_label=['bar1','bar2','bar3']
4 x_pos=list(range(len(bar_label)))
5 plt.bar(x_pos,mean_values,yerr=variance,alpha=0.5)
6 max_y=max(zip(mean_values,variance))
7 plt.ylim([0,(max_y[0]+max_y[1])*1.2])
8 plt.ylabel('variable y')
9 plt.xticks(x_pos,bar_label)
10 plt.show()
```

画图并填充

```
1 x=np.random.randn(100).cumsum()
2 y=np.linspace(0,10,100)
3 fig,ax=plt.subplots()
4 ax.fill_between(x,y,color='lightblue')
5 plt.show()
```

案例

案例1

```
1 x=np.linspace(0,10,200)
2 y1=2*x+1
3 y2=3*x+1.2
4 y_mean=0.5*x*np.cos(2*x)+2.5*x+1.1
5 fig,ax=plt.subplots()
6 ax.fill_between(x,y1,y2,color='red')
7 ax.plot(x,y_mean,color='black')
8 plt.show()
```

案例2

```
1 x1=np.array([1,2,3])
2 x2=np.array([2,2,3])
3 bar_labels=['bar1','bar2','bar3']
4 ; 设定figure大小
5 fig=plt.figure(figsize=(8,6))
6 y_pos=np.arange(len(x1))
7 y_pos=[x for x in y_pos]
8 plt.barh(y_pos,x1,color='g',alpha=0.5)
9 plt.barh(y_pos,-x2,color='b',alpha=0.5)
10 plt.xlim(-max(x2)-1,max(x1)+1)
11 plt.ylim(-1,len(x1)+1)
12 plt.show()
```

案例3:显示多个指标

```
1 green_data=[1,2,3]
2 blue_data=[3,2,1]
3 red_data=[2,3,3]
4 labels=['group1', 'group2', 'group3']
5 pos=list(range(len(green_data)))
6 width=0.2
7 fig,ax=plt.subplots(figsize=(8,6))
8 plt.bar(pos,green_data,width,alpha=0.5,color='g',label=labels[0])
9 plt.bar([p+width for p in pos],blue_data,width,alpha=0.5,color='b',label=labels[1])
10 plt.bar([p+width*2 for p in pos],red_data,width,alpha=0.5,color='r',label=labels[2])
11 plt.show()
```

案例4

```
1 data=range(200,225,5)
2 bar_labels=['a','b','c','d','e']
3 fig=plt.figure(figsize=(10,8))
4 y_pos=np.arange(len(data))
5 plt.yticks(y_pos,bar_labels,fontsize=16)
6 bars=plt.barh(y_pos,data,alpha=0.8,color='g')
7 plt.vlines(min(data),-1,len(data)+0.5,linestyle='dashed')
8 for b,d in zip(bars,data) :
9
10     plt.text(b.get_width()+b.get_width()*0.05,b.get_y()+b.get_height()/2,'{0:.2%}'.format(d/min(data)))
11 plt.show()
```

合图

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 tang_data=[np.random.normal(0,std,100) for std in range(1,8)]
4 fig=plt.figure(figsize=(8,6))
5 plt.boxplot(tang_data,notch=False,sym='s',vert=True)
6 plt.xlabel('x')
7 plt.title('box plot')
8 plt.show()
```

风格

查看可用风格

```
1 plt.style.available
2 plt.style.use('风格名称')
3 plt.style.use(['风格名称1','风格名称2'])
```

colormap

```
1 import matplotlib.pyplot as plt
2 import matplotlib.colors as col
3 import matplotlib.cm as cm
4 mean_value=range(10,18)
5 x_pos=range(len(mean_value))
6 cmap1=cm.ScalarMappable(col.Normalize(min(mean_value),max(mean_value),cm.hot))
7 cmap2=cm.ScalarMappable(col.Normalize(0,20,cm.hot))
8 plt.subplot(121)
9 plt.bar(x_pos,mean_value,color=cmap1.to_rgba(mean_value))
10
11 plt.subplot(122)
12 plt.bar(x_pos,mean_value,color=cmap2.to_rgba(mean_value))
13
14 plt.show()
```

样式填充

```
1 import matplotlib.pyplot as plt
2 patterns=('-', '+', 'x', '\\', '*', 'o', 'O', '.')
3 fig=plt.gca()
4 mean_value=range(1, len(patterns)+1)
5 x_pos=list(range(len(mean_value)))
6 bars = plt.bar(x_pos, mean_value, color='white')
7 for bar, pattern in zip(bars, patterns) :
8     bar.set_hatch(pattern)
9 plt.show()
```